北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

2026

德以明理·学以精工

基于无人机的复杂场景中车牌识别方法研究

Dr. Yujie Fang (方宇杰)
TA：Junming Wu (吴浚铭)

BIT

北京理工大学

2024年11月

使用OpenCV对图像进行基本操作与处理

1.1

图像处理基本操作

在图像处理过程中，读取图像、显示图像、保存图像是最基本的操作。本节将简单介绍这几项基本操作。

1.2.1 读取图像

OpenCV提供了函数cv2.imread）来读取图像，该函数支持各种静态图像格式。该函数的语法格式为：retval=cv2.imread（filename[，flags]）式中：

- retval是返回值，其值是读取到的图像。如果未读取到图像，则返回“None”。

- filename表示要读取的图像的完整文件名。

- flags是读取标记。该标记用来控制读取文件的类型，具体如表1-l所示。表1-1中的第一列参数与第三列数值是等价的。例如cv2.IMREADUNCHANGED-1，在设置参数时，既可以使用第一列的参数值，也可以采用第三列的数值。

图像处理基本操作

例如，想要读取当前目录下文件名为lena.bmp的图像，并保持按照原有格式读入，则使用的语句为：

lena=cv2.imread（"lena.bmp"，-1）

需要注意，上述程序要想正确运行，首先需要导入cv2模块，大多数常用的OpenCV函数都在cv2模块内。与cv2模块所对应的cv模块代表传统版本的模块。这里的cv2模块并不代表该模块是专门针对OpenCV2版本的，而是指该模块引入了一个改善的API接口。在cv2模块内部采用了面向对象的编程方式，而在cv模块内更多采用的是面向过程的编程方式。

本书中所使用的模块函数都是cv2模块函数，为了方便理解，在函数名前面加了“cv2.”。

但是如果函数名出现在标题中，那么希望突出的是该函数本身，所以未加“cv2.”。

【例1.1】使用cv2.imread()函数读取一幅图像。

根据题目要求，编写代码如下：

```
import cv21ena=cv2.imread（"lenacolor.png"）
print（lena）
```

1.2.2 显示图像

OpenCV提供了多个与显示有关的函数，下面对常用的几个进行简单介绍。

1.namedWindow函数

函数cv2.namedWindow()用来创建指定名称的窗口，其语法格式为：

None=cv2.namedwindow（winname）

式中：

• winname是要创建的窗口的名称。

例如，下列语句会创建一个名为lesson的窗口：cv2.namedWindow（"lesson"）

2.imshow函数

函数cv2.imshow()用来显示图像，其语法格式为：None=cv2.imshow（winname，mat）

式中：

•winname是窗口名称。

•mat是要显示的图像。

图像处理基本操作

【例1.2】

在一个窗口内显示读取的图像。根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread（"lena.bmp"）
cv2.namedWindow（"lesson"）
cv2.imshow（"lesson"，lena）
```

在本程序中，首先通过cv2.imread()函数读取图像lena.bmp，接下来通过cv2.namedWindow()函数创建一个名为lesson的窗口，最后通过cv2.imshow()函数在窗口lesson内显示图像lena.bmp。

运行上述程序，得到的运行结果如图1-3所示。



图1-3 例1-2程序运行结果

1.2.3保存图像

OpenCV提供了函数cv2.imwrite），用来保存图像，该函数的语法格式为：

retval=cv2.imwrite（filename，img[，params])

式中：

- retval是返回值。如果保存成功，则返回逻辑值真（True）；如果保存不成功，则返回逻辑值假（False）。
- filename是要保存的目标文件的完整路径名，包含文件扩展名。
- img是被保存图像的名称。
- params是保存类型参数，是可选的。

图像处理基本操作

【例1.7】编写一个程序，将读取的图像保存到当前目录下。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena.bmp")
r=cv2.imwrite("result.bmp",lena)
```

上述程序会先读取当前目录下的图像lena.bmp，生成它的一个副本图像，然后将该图像以名称result.bmp存储到当前目录下。

1.2

像素处理

像素处理

像素是图像构成的基本单位，像素处理是图像处理的基本操作，可以通过位置索引的形式对图像内的元素进行访问、处理。

1.二值图像与灰度图像

需要说明的是，在OpenCV中，最小的数据类型是无符号的8位数。因此，在OpenCV中实际上并没有二值图像这种数据类型，二值图像经常是通过处理得到的，然后使用0表示黑色，使用255表示白色。

可以将二值图像理解为特殊的灰度图像，这里仅以灰度图像为例讨论像素的读取和修改。

通过2.1节的分析可知，可以将图像理解为一个矩阵，在面向Python的OpenCV（OpenCV for Python）中，图像就是Numpy库中的数组。一个OpenCV灰度图像是一个二维数组，可以使用表达式访问其中的像素值。例如，可以使用image[0，0]访问图像image第0行第0列位置上的像素点。第0行第0列位于图像的左上角，其中第1个索引表示第0行，第2个索引表示第0列。

为了方便理解，我们首先使用Numpy库来生成一个8×8大小的数组，用来模拟一个黑色图像，并对其进行简单处理。

【例2.2】读取一个灰度图像，并对其像素进行访问、修改。

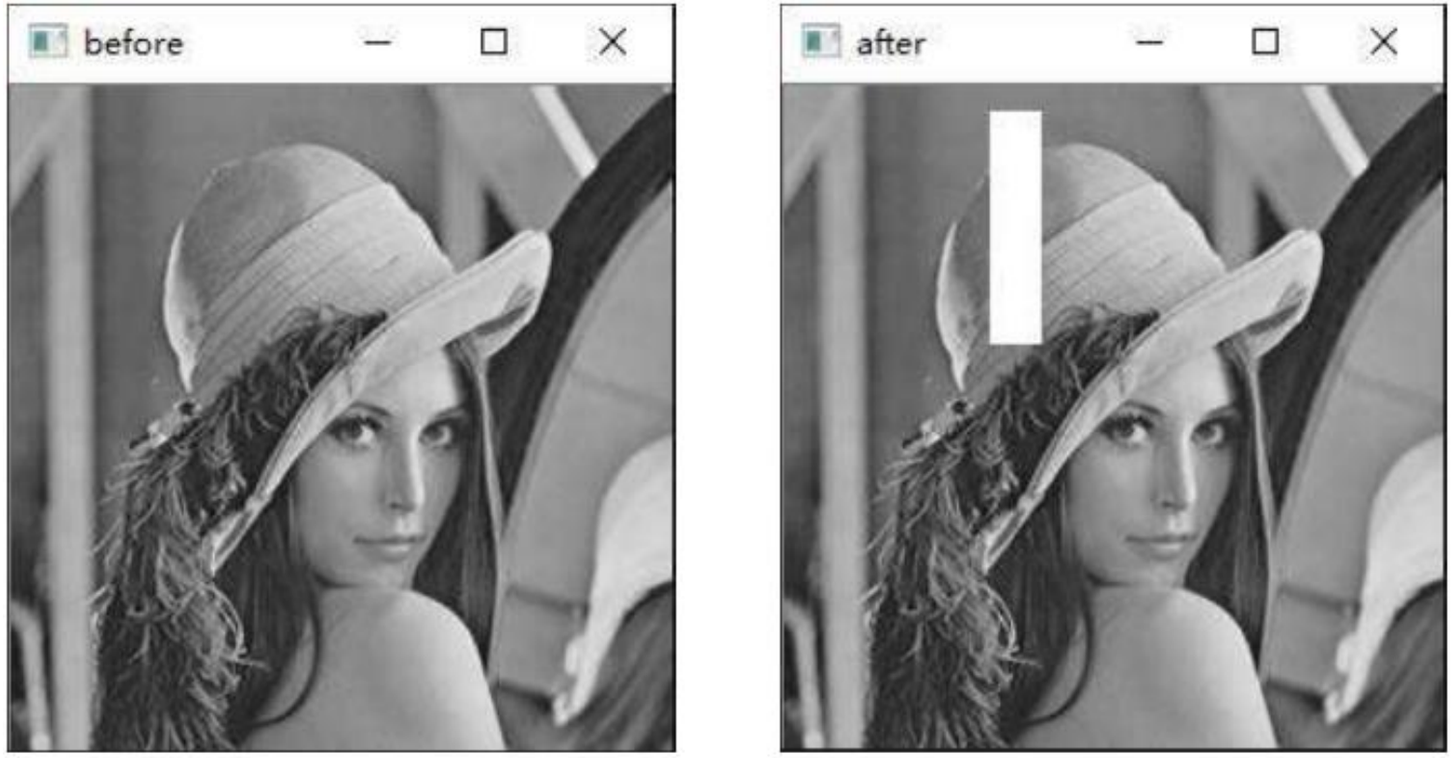
根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("bear.jpeg",0)
cv2.imshow("before",img)
for i in range(10,100):
    for j in range(80,100):
        img[i,j]=255
cv2.imshow("after",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，使用了一个嵌套循环语句，将图像img中“第10行到99行”与“第80列到99列”交叉区域内的像素值设置为255。从图像img上来看，该交叉区域被设置为白色。

运行程序，结果如图2-6所示，其中：

读取的
原始图像



经过修改后
的图像

图2-6 像素修改示例

2.彩色图像

RGB模式的彩色图像在读入OpenCV内进行处理时，会按照行方向依次读取该RGB图像的B通道、G通道、R通道的像素点，并将像素点以行为单位存储在ndarray的列中。例如，有一幅大小为R行×C列的原始RGB图像，其在OpenCV内以BGR模式的三维数组形式存储，如图2-7所示。

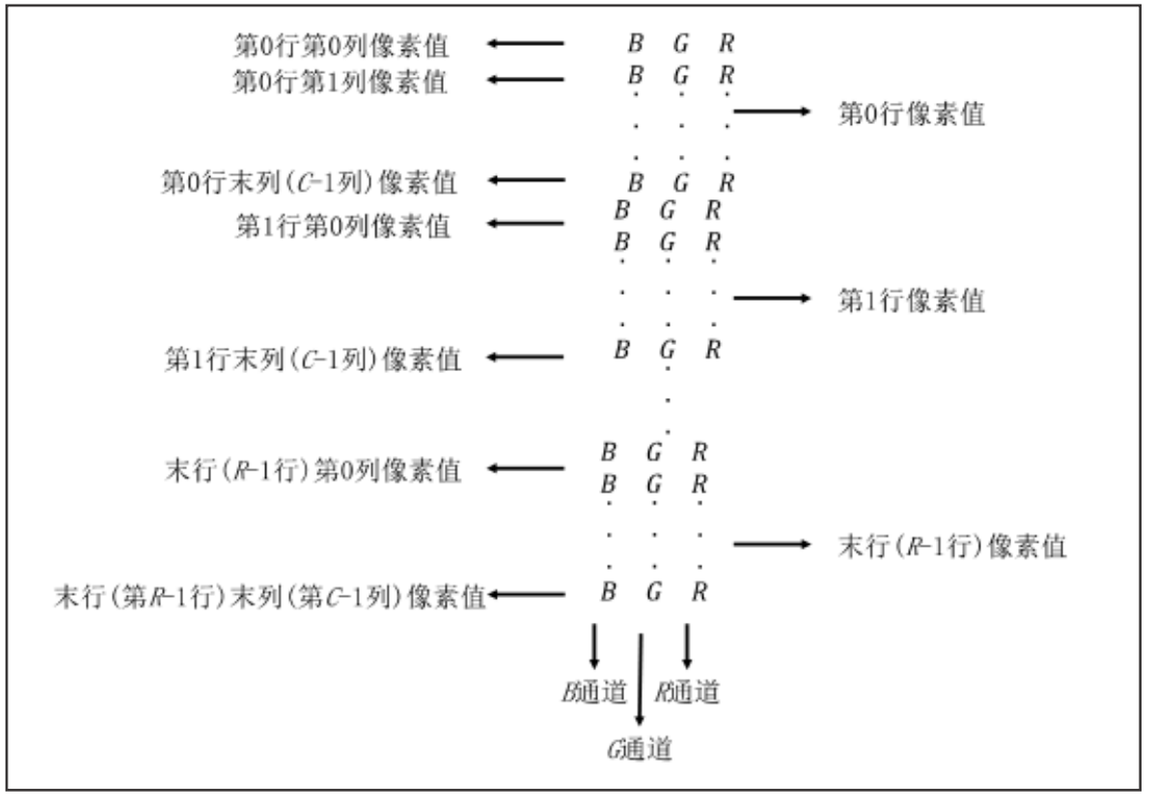


图2-7 RGB图像以三维数组形式存储的情况说明

像素处理

可以使用表达式访问数组内的值。例如，可以使用image[0，0，0]访问图像image的B通道内的第0行第0列上的像素点，式中：

- 第1个索引表示第0行
- 第2个索引表示第0列
- 第3个索引表示第0个颜色通道

根据上述分析可知，假设有一个红色（其R通道值为255，G通道值为0，B通道值为0）图像，不同的访问方式得到的值如下。

- img[0，0]：访问图像img第0行第0列像素点的BGR值。图像是BGR格式的，得到的数值为[0，0，255]。
- img[0，0，0]：访问图像img第0行第0列第0个通道的像素值。图像是BGR格式的，所以第0个通道是B通道，会得到B通道内第0行第0列的位置所对应的值0。
- img[0，0，1]：访问图像img第0行第0列第1个通道的像素值。图像是BGR格式的，所以第1个通道是G通道，会得到G通道内第0行第0列的位置所对应的值0。
- img[0，0，2]：访问图像img第0行第0列第2个通道的像素值。图像是BGR格式的，所以第2个通道是R通道，会得到R通道内第0行第0列的位置所对应的值255。



感兴趣区域（ROI）

感兴趣区域（ROI）

在图像处理过程中，我们可能会对图像的某一个特定区域感兴趣，该区域被称为感兴趣区域（Region of Interest，ROI）。在设定感兴趣区域ROI后，就可以对该区域进行整体操作。例如，将一个感兴趣区域A赋值给变量B后，可以将该变量B赋值给另外一个区域C，从而达到在区域C内复制区域A的目的。

例如，在图2-16中，假设当前图像的名称为img，图中的数字分别表示行号和列号。那么，图像中的黑色ROI可以表示为img[200：400，200：400]。

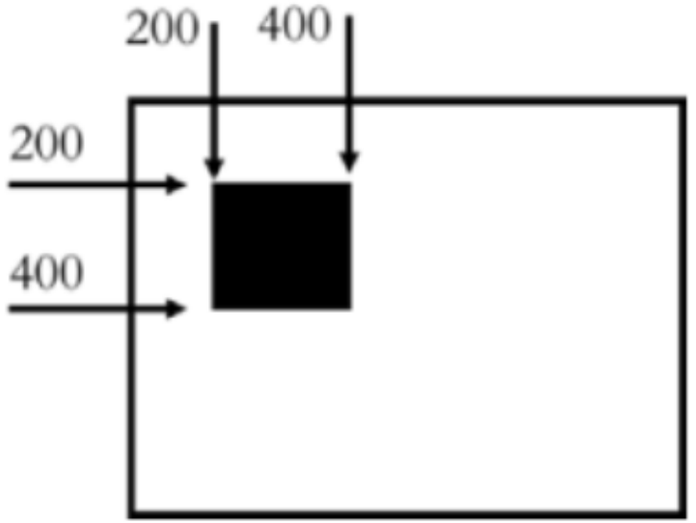


图2-16 ROI示例

感兴趣区域（ROI）

【例2.13】获取图像lena的脸部信息，并将其显示出来。

根据题目要求，编写代码如下：

```
import cv2
a=cv2.imread("bear.jpeg",cv2.IMREAD_UNCHANGED)#cv2.IMREAD_UNCHANGED按原样返回加载的图像
face=a[220:400,250:350]
cv2.imshow("original",a)
cv2.imshow("face",face)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，通过face=a[220：400，250：350]获取了一个ROI，并使用函数cv2.imshow）将其显示了出来。

感兴趣区域（ROI）

运行上述程序，会得到如图2-18所示的结果，其中左图是lena的原始图像，右图是从lena图像中获取的脸部图像。

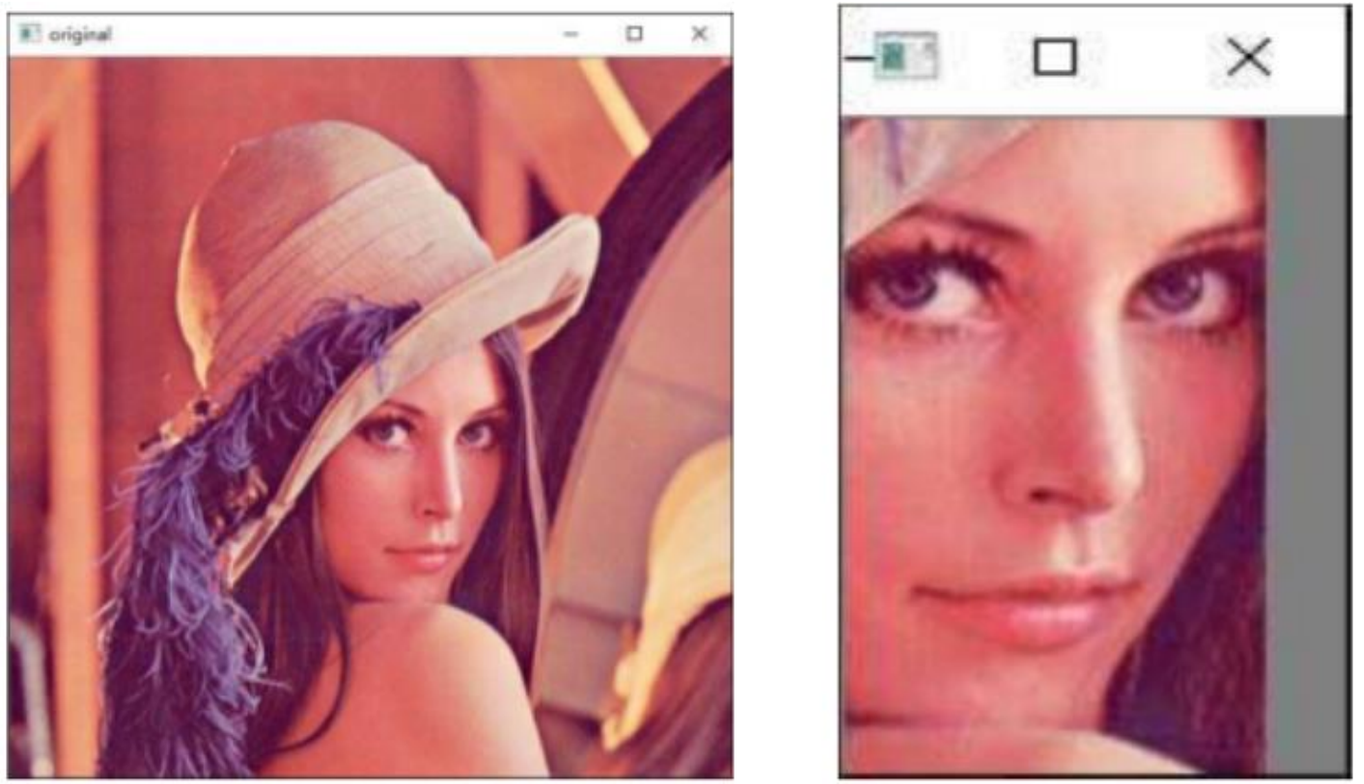


图2-18 【例2.13】程序的运行结果

感兴趣区域（ROI）

【例2.15】将一幅图像内的RO1复制到另一幅图像内。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("bear.jpeg",cv2.IMREAD_UNCHANGED)
dollar=cv2.imread("dollar.jpeg",cv2.IMREAD_UNCHANGED)
cv2.imshow("lena",lena)
cv2.imshow("dollar",dollar)
face=lena[220:400,250:350]
dollar[160:340,200:300]=face
cv2.imshow("result",dollar)
cv2.waitKey()
cv2.destroyAllWindows()
```

1.4

通道操作

在RGB图像中，图像是由R通道、G通道、B通道三个通道构成的。需要注意的是，在OpenCV中，通道是按照B通道→G通道→R通道的顺序存储的。

在图像处理过程中，可以根据需要对通道进行拆分和合并。本节就来介绍如何对通道进行拆分和合并。

2.5.1读取图像

针对RGB图像，可以分别拆分出其R通道、G通道、B通道。在OpenCV中，既可以通过索引的方式拆分通道，也可以通过函数的方式拆分通道。

1.通过索引拆分

通过索引的方式，可以直接将各个通道从图像内提取出来。例如，针对OpenCV内的BGR图像img，如下语句分别从中提取了B通道、G通道、R通道。

```
b=img[：，：，0]
g=img[：，：，1]
r=img[：，：，2]
```

【例2.16】编写程序，演示图像通道拆分及通道值改变对彩色图像的影响。

根据题目要求，编写代码如下：

通道操作


```
import cv2
lena=cv2.imread("bear.jpeg")
cv2.imshow("lena1",lena)
b=lena[:,:,0]
g=lena[:,:,1]
r=lena[:,:,2]
cv2.imshow("b",b)
cv2.imshow("g",g)
cv2.imshow("r",r)
lena[:,:,0]=0
cv2.imshow("lenab0",lena)
lena[:,:,1]=0
cv2.imshow("lenab0g0",lena)
cv2.waitKey()
cv2.destroyAllWindows()
```

2.5.2通道合并

通道合并是通道拆分的逆过程，通过合并通道可以将三个通道的灰度图像构成一幅彩色图像。函数cv2.merge（）可以实现图像通道的合并，例如有B通道图像b、G通道图像g和R通道图像r，使用函数cv2.merge）可以将这三个通道合并为一幅BGR的三通道彩色图像。其实现的语句为：

bgr=cv2.merge（[b，g，r]）

【例2.18】编写程序，演示使用函数cv2.merge（）合并通道。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("bear.jpeg")
b,g,r=cv2.split(lena)
bgr=cv2.merge([b,g,r])
rgb=cv2.merge([r,g,b])
```

通道操作

运行上述程序，得到如图2-23所示的图像，其中：

- 左图是原始图像lena。
- 中间的图是lena图像经过通道拆分、合并后得到的BGR通道顺序的彩色图像bgr。
- 右图是lena图像经过通道拆分、合并后得到的RGB通道顺序的彩色图像rgb。

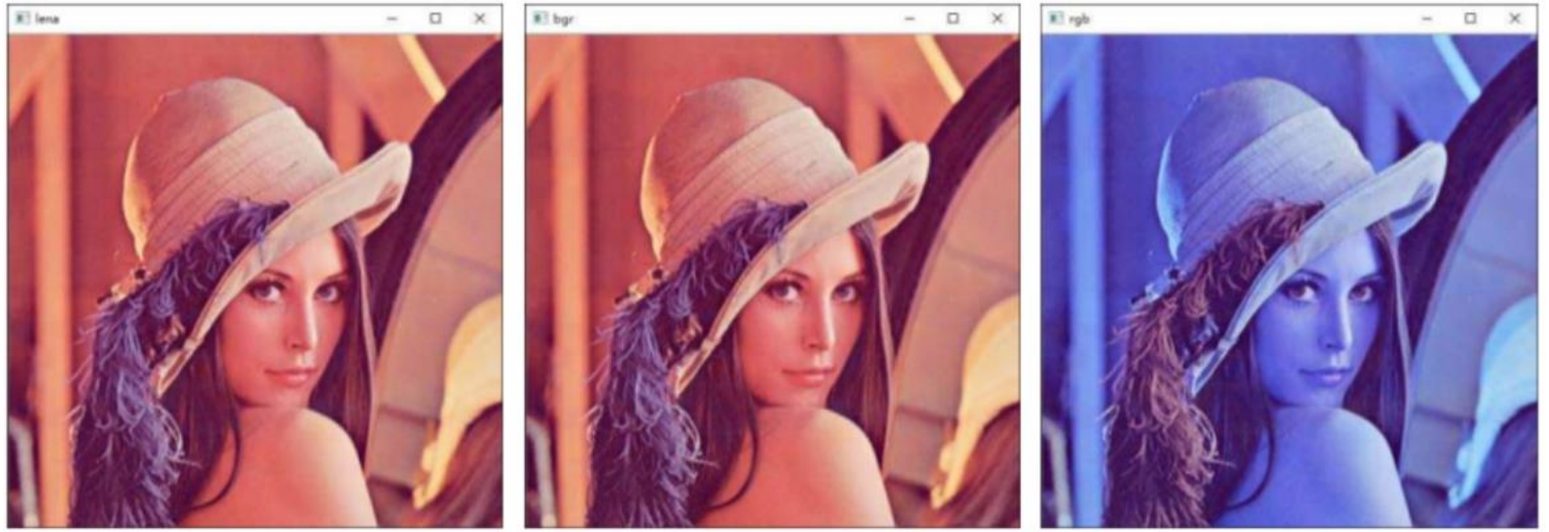


图2-23 【例2.18】程序的运行结果

通过本例可以看出，改变通道顺序后，图像显示效果会发生变化。

1.5

类型转换实例

本节介绍几种常用的色彩空间转换实例，以帮助大家更好地理解应该如何使用函数cv2.cvtColor()。

类型转换函数

在 OpenCV 内，我们使用 cv2.cvtColor()函数实现色彩空间的变换。该函数能够实现多个色彩空间之间的转换。其语法格式为：

dst = cv2.cvtColor(src, code [, dstCn])

式中：

- dst 表示输出图像，与原始输入图像具有同样的数据类型和深度。
- src 表示原始输入图像。可以是8 位无符号图像、16 位无符号图像，或者单精度浮点数等。
- code 是色彩空间转换码，表 4-2 展示了其枚举值。
- dstCn 是目标图像的通道数。如果参数为默认的 0，则通道数自动通过原始输入图像和code 得到。

表 4-2 枚举值

值	备注
cv2.COLOR_BGR2BGRA	为 BGR 或 RGB 图像添加 alpha 通道（alpha 是透明属性）
cv2.COLOR_RGB2RGBA	
cv2.COLOR_BGRA2BGR	从 BGR 或 RGB 通道内删除 alpha 通道
cv2.COLOR_RGBA2RGB	

4.3.2图像处理实例

本节将通过具体实例介绍如何使用函数 cv2.cvtcolor()来处理图像类型的转换。例 【例 4.4 】将图像在 BGR 模式和灰度图像之间相互转换。

根据题目要求，设计程序如下：

```
import cv2
lena=cv2.imread("lenacolor.png")
gray=cv2.cvtColor(lena,cv2.COLOR_BGR2GRAY)
rgb=cv2.cvtColor(gray,cv2.COLOR_GRAY2BGR)
#==========打印 shape===========
print("lena.shape=",lena.shape)
print("gray.shape=",gray.shape)
print("rgb.shape=",rgb.shape)
#==========显示效果===========
cv2.imshow("lena",lena)
cv2.imshow("gray",gray)
```

类型转换实例


```
cv2.imshow("rgb",rgb)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，会显示各个图像的 shape 属性：

```
lena.shape= (512, 512, 3)
gray.shape= (512, 512)
rgb.shape= (512, 512, 3)
```

通过其 shape 属性，可以看到图像在转换前后的色彩空间变化情况。

同时，程序会分别显示原始彩色图像、灰度图像、RGB 图像。

需要注意，在通过“rgb=cv2.cvtColor(gray,cv2.COLOR_GRAY2BGR)”得到的 RGB 图像中，B 通道、G 通道、R通道的值都是一样的，所以其看起来仍是灰度图像。由于本书是黑白（灰度）印刷，无法观察彩色效果，所以，这里省略了运行结果中图像的展示。请大家在 Python 内运行上述程序，观察实际效果。

类型转换实例

【例 4.5 】将图像从 BGR 模式转换为 RGB 模式。

根据题目要求，设计程序如下：

```
import cv2
lena=cv2.imread("lenacolor.png")
rgb = cv2.cvtColor(lena, cv2.COLOR_BGR2RGB)
cv2.imshow("lena",lena)
cv2.imshow("rgb",rgb)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，会显示如图 4-3 所示的运行结果。

类型转换实例

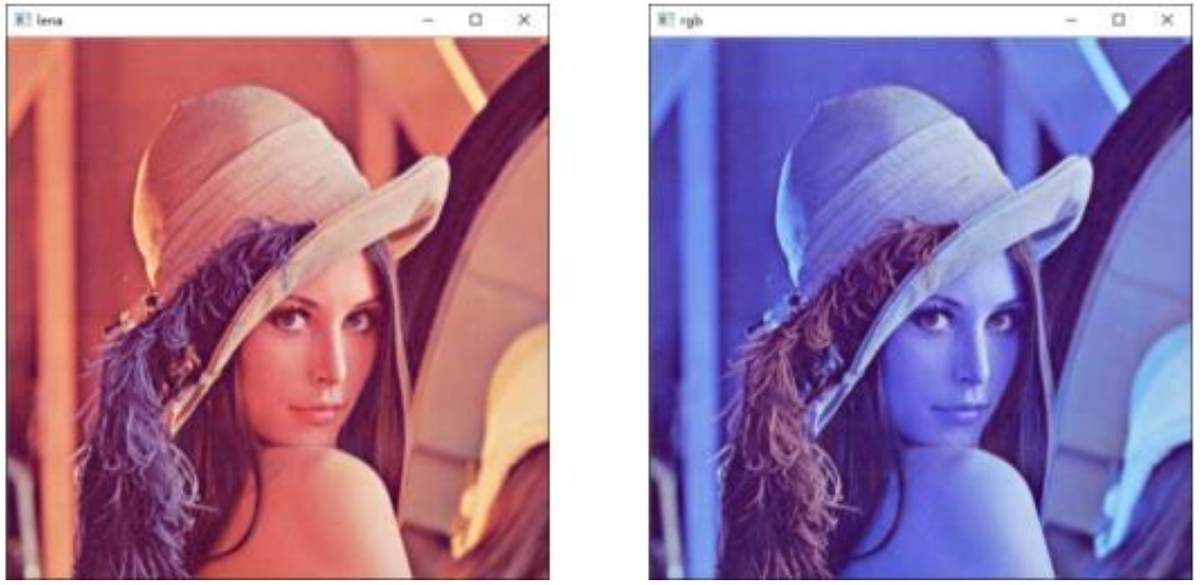


图 4-3 【例 4.5】程序的运行结果

在图 4-3 中，左图是 BGR 通道顺序的图像，右图是 RGB 通道顺序的图像。可以看到，读取的lena图像在BGR模式下正常显示，将其调整为RGB通道顺序后，显示的图像呈现浅蓝色。

3．显示特定颜色值

【例 4.9 】分别提取 OpenCV 的 logo 图像内的红色、绿色、蓝色。

需要注意，在实际提取颜色时，往往不是提取一个特定的值，而是提取一个颜色区间。例如，在 OpenCV 中的 HSV 模式内，蓝色在 H 通道内的值是 120。在提取蓝色时，通常将“蓝色值 120”附近的一个区间的值作为提取范围。该区间的半径通常为 10 左右，例如通常提取[120−10,120+10]范围内的值来指定蓝色。

相比之下，HSV 模式中 S 通道、V 通道的值的取值范围一般是[100,255]。这主要是因为，当饱和度和亮度太低时，计算出来的色调可能就不可靠了。

根据上述分析，各种颜色的 HSV 区间值分布在[H−10,100,100]和[H+10,255,255]之间。因此，各种颜色值的范围为：

- **蓝色：值分布在[110,100,100]和[130,255,255]之间。**
- **绿色：值分布在[50,100,100]和[70,255,255]之间。**
- **红色：值分布在[0,100,100]和[10,255,255]之间。**

HSV色彩空间讨论

运行程序，结果如图 4-4 所示，其中：

- 图(a)是原始图像。
- 图(b)是从图(a)中提取得到的蓝色部分。
- 图(c)是从图(a)中提取得到的绿色部分。
- 图(d)是从图(a)中提取得到的红色部分。

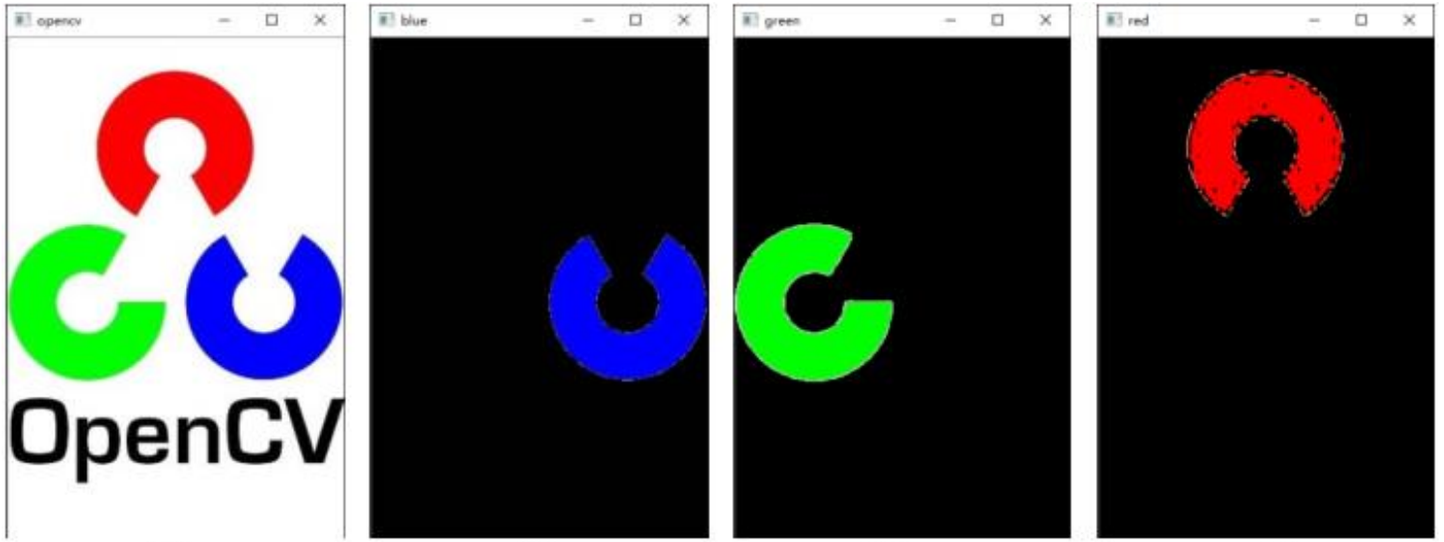


图 4-4 【例 4.9】程序的运行结果

1.6

阈值处理

阈值处理

阈值处理是指剔除图像内像素值高于一定值或者低于一定值的像素点。例如，设定阈值为127，然后：

- 将图像内所有像素值大于 127 的像素点的值设为 255。
- 将图像内所有像素值小于或等于 127 的像素点的值设为 0。

通过上述方式能够得到一幅二值图像，如图 6-1 所示，按照上述阈值处理方式将一幅灰度图像处理为一幅二值图像，有效地实现了前景和背景的分离。



图 6-1 阈值处理示例

OpenCV 提供了函数 cv2.threshold()和函数 cv2.adaptiveThreshold()，用于实现阈值处理。

阈值处理

OpenCV 3.0 使用 cv2.threshold()函数进行阈值化处理，该函数的语法格式为：

retval, dst = cv2.threshold(src, thresh, maxval, type)

式中：

- retval 代表返回的阈值。
- dst 代表阈值分割结果图像，与原始图像具有相同的大小和类型。
- src 代表要进行阈值分割的图像，可以是多通道的，8 位或 32 位浮点型数值。
- thresh 代表要设定的阈值。
- maxval 代表当 type 参数为 THRESH_BINARY 或者 THRESH_BINARY_INV 类型时，需要设定的最大值。
- type 代表阈值分割的类型，具体类型值如表 6-1 所示。

阈值处理

表 6-1 阈值分割类型

类型	定义
cv2.THRESH_BINARY	$\mathrm{dst}(x,y)=\begin{cases}\mathrm{maxval}, & \mathrm{src}(x,y)>\mathrm{thresh}\\ 0, & \text{其他情况}\end{cases}$
cv2.THRESH_BINARY_INV	$\mathrm{dst}(x,y)=\begin{cases}0, & \mathrm{src}(x,y)>\mathrm{thresh}\\ \mathrm{maxval}, & \text{其他情况}\end{cases}$
cv2.THRESH_TRUNC	$\mathrm{dst}(x,y)=\begin{cases}\mathrm{thresh}, & \mathrm{src}(x,y)>\mathrm{thresh}\\ 0, & \text{其他情况}\end{cases}$
cv2.THRESH_TOZERO_INV	$\mathrm{dst}(x,y)=\begin{cases}0, & \mathrm{src}(x,y)>\mathrm{thresh}\\ \mathrm{src}(x,y), & \text{其他情况}\end{cases}$
cv2.THRESH_TOZERO	$\mathrm{dst}(x,y)=\begin{cases}\mathrm{src}(x,y), & \mathrm{src}(x,y)>\mathrm{thresh}\\ 0, & \text{其他情况}\end{cases}$
cv2.THRESH_MASK	掩码
cv2.THRESH_OTSU	标记，使用 Otsu 算法时的可选阈值参数
cv2.THRESH_TRIANGLE	标记，使用 Triangle 算法时的可选阈值参数

上述公式相对抽象，可以将其可视化，具体如图 6-2 所示。

阈值处理

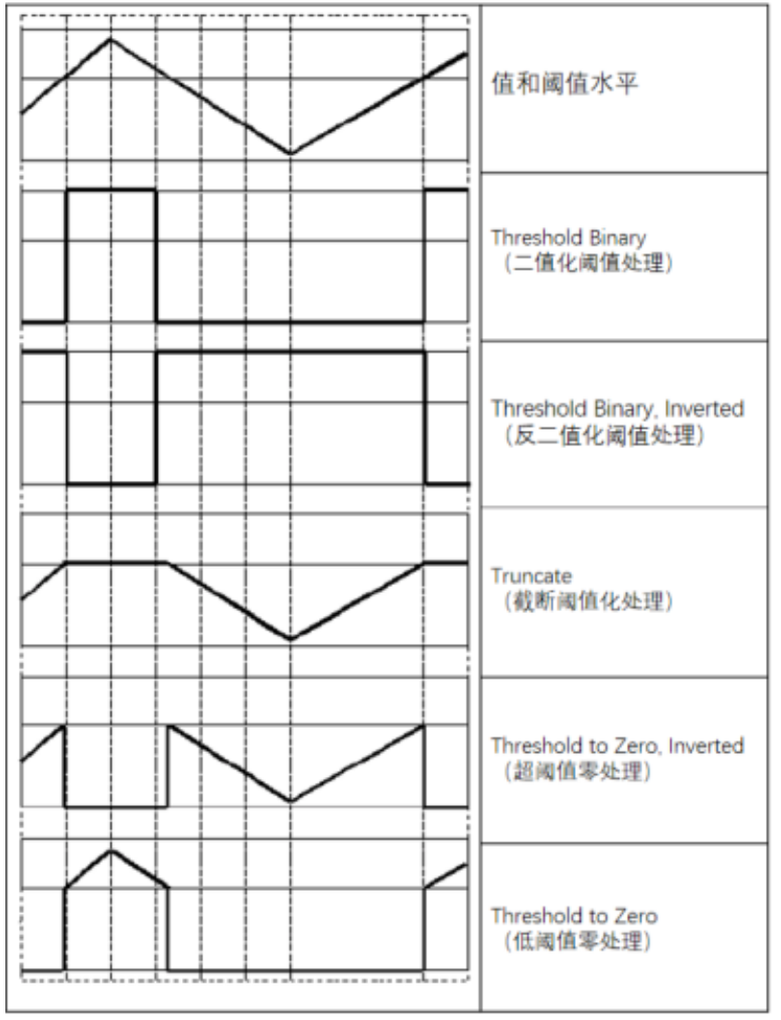


图 6-2 阈值分割类型可视化表示

6.1.1二值化阈值处理（cv2.THRESH_BINARY）

二值化阈值处理会将原始图像处理为仅有两个值的二值图像，其示意图如图 6-3 所示。其针对像素点的处理方式为：

- 对于灰度值大于阈值thresh的像素点，将其灰度值设定为 最大值。
- 对于灰度值小于或等于阈值thresh的像素点，将其灰度值设定为 0。

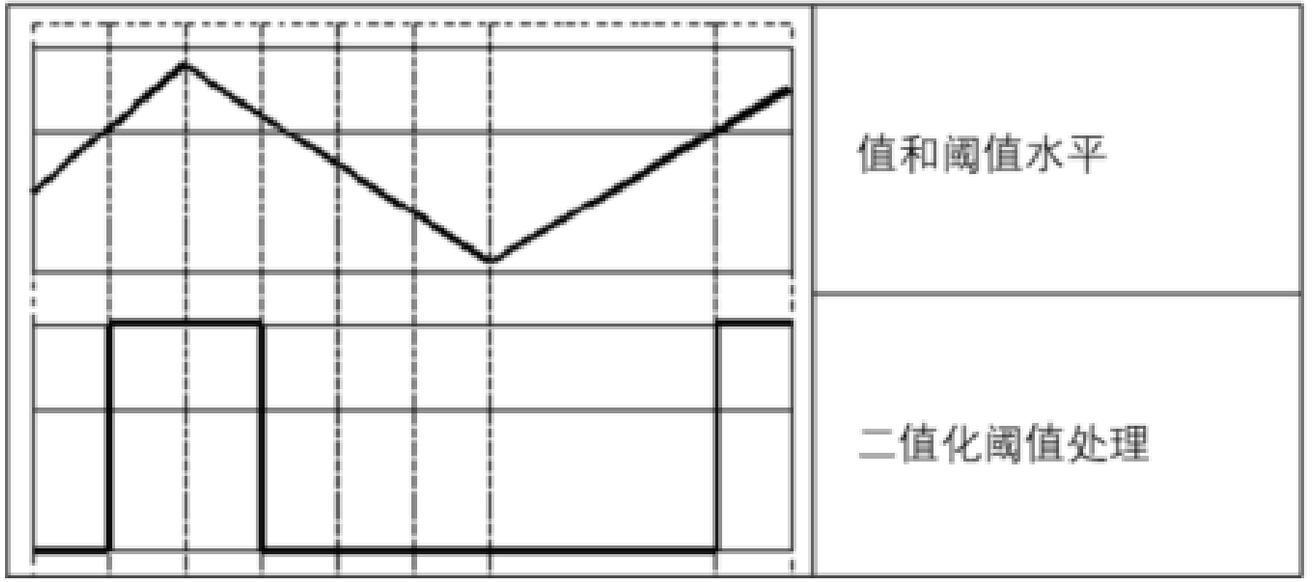


图 6-3 二值化阈值处理

阈值处理

如果使用表达式表示，其目标值的产生规则为：

$$\mathrm{dst}(x,y)=\begin{cases}\mathrm{maxval}, & \mathrm{src}(x,y)>\mathrm{thresh}\\ 0, & \text{其他情况}\end{cases}$$

式中，thresh是选定的特定阈值。

在 8 位图像中，最大值是 255。因此，在对 8 位灰度图像进行二值化时，如果将阈值设定为 127，那么：

- 所有大于 127 的像素点会被处理为 255。
- 其余值会被处理为 0。

为了方便，在后续说明中，我们都以 8 位图像为例，即像素值最大值为 255。

阈值处理

【例 6.2 】使用函数 cv2.threshold()对图像进行二值化阈值处理。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp")
t,rst=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
cv2.imshow("img",img)
cv2.imshow("rst",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

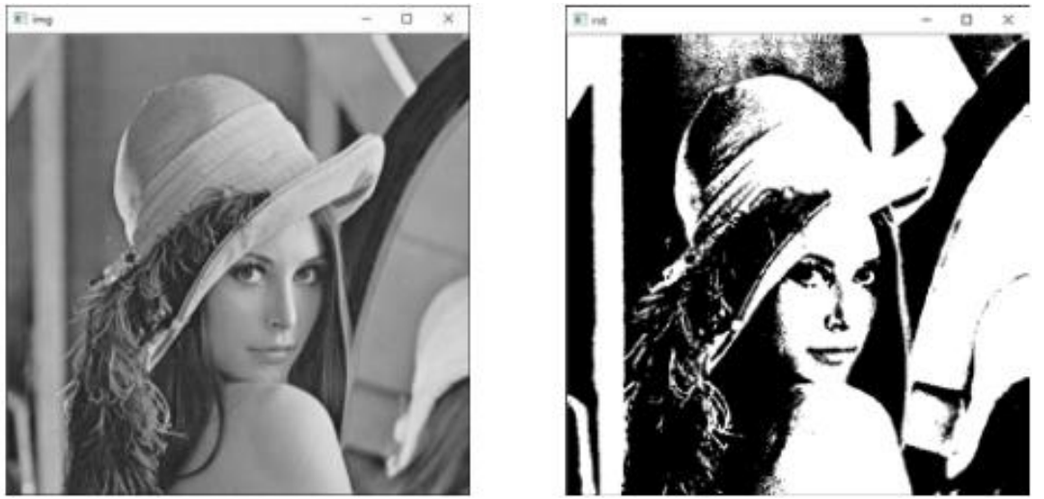


图 6-4 二值化阈值处理结果

运行程序，结果如图 6-4 所示，左图是原始图像，右图是二值化阈值处理结果。

自适应阈值处理

对于色彩均衡的图像，直接使用一个阈值就能完成对图像的阈值化处理。但是，有时图像的色彩是不均衡的，此时如果只使用一个阈值，就无法得到清晰有效的阈值分割结果图像。

有一种改进的阈值处理技术，其使用变化的阈值完成对图像的阈值处理，这种技术被称为自适应阈值处理。在进行阈值处理时，自适应阈值处理的方式通过计算每个像素点周围临近区域的加权平均值获得阈值，并使用该阈值对当前像素点进行处理。与普通的阈值处理方法相比，自适应阈值处理能够更好地处理明暗差异较大的图像。

OpenCV 提供了函数 cv2.adaptiveThreshold()来实现自适应阈值处理，该函数的语法格式为：

```
dst = cv.adaptiveThreshold( src, maxValue, adaptiveMethod, thresholdType, blockSize, C )
```

式中：

- dst 代表自适应阈值处理结果。
- src 代表要进行处理的原始图像。需要注意的是，该图像必须是 8 位单通道的图像。
- maxValue 代表最大值。

自适应阈值处理

【例 6.11 】对一幅图像分别使用二值化阈值函数 cv2.threshold()和自适应阈值函数cv2.adaptiveThreshold()进行处理，观察处理结果的差异。

根据题目要求，编写代码如下：

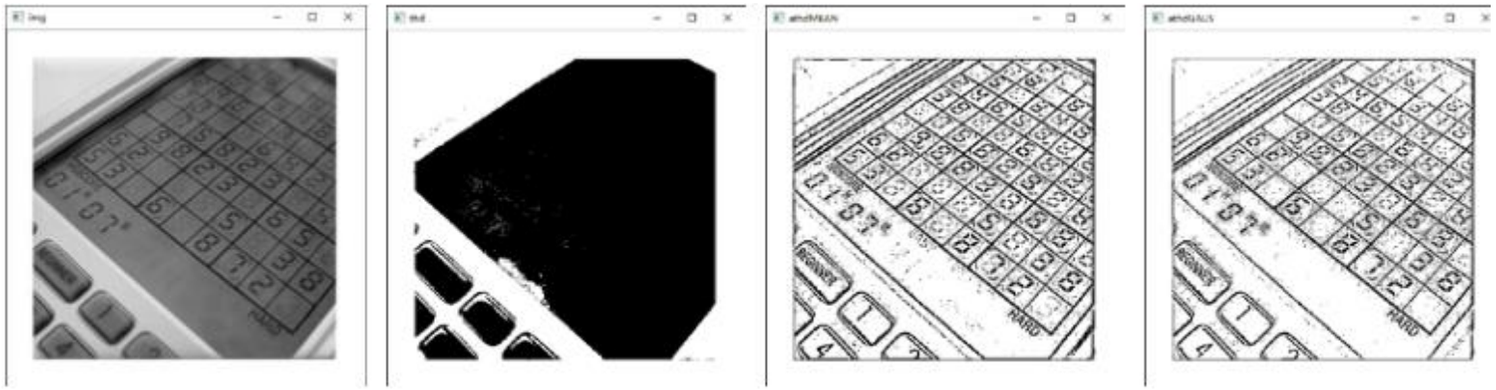
```
import cv2
img=cv2.imread("computer.jpg",0)
t1,thd=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
athdMEAN=cv2.adaptiveThreshold(img,255,cv2.ADAPTIVE_THRESH_MEAN_C,cv2.THRESH_BINARY,5,3)
athdGAUS=cv2.adaptiveThreshold(img,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,5,3)
cv2.imshow("img",img)
cv2.imshow("thd",thd)
cv2.imshow("athdMEAN",athdMEAN)
cv2.imshow("athdGAUS",athdGAUS)
cv2.waitKey()
cv2.destroyAllWindows()
```

自适应阈值处理

运行程序，结果如图 6-13 所示，其中：

- 图(a)是原始图像。
- 图(b)是二值化阈值处理结果。
- 图(c)是自适应阈值采用方法 cv2.ADAPTIVE_THRESH_MEAN_C 的处理结果。
- 图(d)是自适应阈值采用方法 cv2.ADAPTIVE_THRESH_GAUSSIAN_C 的处理结果。

通过对比普通的阈值处理与自适应阈值处理可以发现，自适应阈值处理保留了更多的细节信息。在一些极端情况下，普通的阈值处理会丢失大量的信息，而自适应阈值处理可以得到效果更好的二值图像。



(a) (b) (c) (d)

图 6-13 【例 6.11】程序的运行结果

Otsu 处理

需要说明的是，在使用 Otsu 方法时，要把阈值设为 0。此时的函数 cv2.threshold()会自动寻找最优阈值，并将该阈值返回。例如，下面的语句让函数 cv2.threshold()采用 Otsu 方法进行阈值分割：

```
t,otsu=cv2.threshold(img,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
```

与普通阈值分割的不同之处在于：

- 参数 type 增加了一个参数值“cv2.THRESH_OTSU”。
- 设定的阈值为 0。
- 返回值 t 是 Otsu 方法计算得到并使用的最优阈值。

需要注意，如果采用普通的阈值分割，返回的阈值就是设定的阈值。例如下面的语句设定阈值为 127，所以最终返回的就是 t=127。

```
t,thd=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
```

Otsu 处理



图 6-14 【例 6.13】程序的运行结果

在本例中，原始图像整体的亮度较高，即有较多的像素值都是大于 127 的，所以：

- 在使用 127 作为阈值进行普通的二值阈值化处理时，得到了大量的白色区域。
- 在使用 Otsu 方法进行处理时，因为通过计算采用了最优阈值，所以得到了较好的处理结果。

在尽量保留图像原有信息的情况下，过滤掉图像内部的噪声，这一过程称为对图像的平滑处理，所得的图像称为平滑图像。例如，图 7-1 是含有噪声的图像，在图像内存在噪声信息，我们通常会通过图像平滑处理等方式去除这些噪声信息。



图 7-1 含有噪声的图像

通过图像平滑处理，可以有效地过滤掉图像内的噪声信息。如图 7-2 所示是对图 7-1 进行图像平滑处理的结果，可以看到原有图像内含有的噪声信息被有效地过滤掉了。



图 7-2 平滑图像

1.7

均值滤波

均值滤波是指用当前像素点周围 N•N 个像素值的均值来代替当前像素值。使用该方法遍历处理图像内的每一个像素点，即可完成整幅图像的均值滤波。

如果针对图像内的每一个像素点都进行上述平滑处理，就能够对整幅图像完成平滑处理，有效地去除图像内的噪声信息。

图像平滑处理的基本原理是，将噪声所在像素点的像素值处理为其周围临近像素点的值的近似值。取近似值的方式很多，本章主要介绍：

- 均值滤波
- 方框滤波
- 高斯滤波
- 中值滤波
- 双边滤波
- 2D 卷积（自定义滤波）

图像平滑处理对应的是英文 Smoothing Images。图像平滑处理通常伴随图像模糊操作，因此图像平滑处理有时也被称为图像模糊处理，图像模糊处理对应的英文是 Blurring Images。

图像滤波是图像处理和计算机视觉中最常用、最基本的操作。图像滤波允许在图像上进行各种各样的操作，因此有时我们也会把图像平滑处理称为图像滤波，图像滤波对应的英文是Images Filtering。

在阅读文献时，针对图像平滑处理，我们可能会遇到各种不同的说法，因为不同的学者对于图像平滑处理的称呼可能不一样。所以，希望大家在学习时不要太纠结这些称呼的不同，而要将注意力集中在如何更好地理解图像算法、如何更好地处理图像上。

7.1.2函数语法

在 OpenCV 中，实现均值滤波的函数是 cv2.blur()，其语法格式为：

```
dst = cv2.blur( src, ksize, anchor, borderType )
```

式中：

- dst 是返回值，表示进行均值滤波后得到的处理结果。
- src 是需要处理的图像，即原始图像。它可以有任意数量的通道，并能对各个通道独立处理。图像深度应该是 CV_8U、CV_16U、CV_16S、CV_32F 或者 CV_64F 中的一种。
- ksize 是滤波核的大小。滤波核大小是指在均值处理过程中，其邻域图像的高度和宽度。

德以明理 学以精工

7.1.3程序示例

【例 7.1】读取一幅噪声图像，使用函数 cv2.blur()对图像进行均值滤波处理，得到去噪图像，并显示原始图像和去噪图像。

根据题目要求，编写代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png") #读取待处理图像
r=cv2.blur(o,(5,5)) #使用 blur 函数处理
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```



图 7-13 中值滤波示例

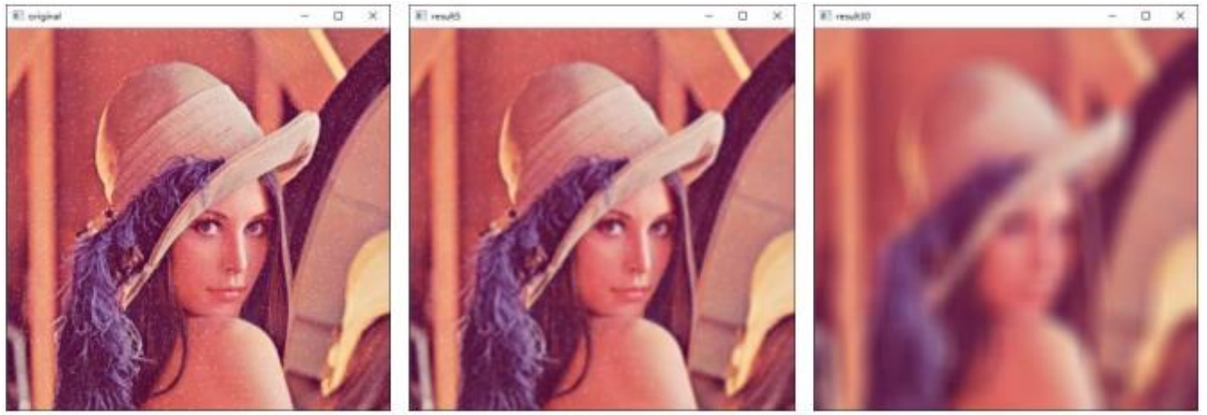
运行上述程序后，会分别显示噪声图像（左图）和去噪图像（右图），如图 7-13 所示。

均值滤波

运行上述程序，分别显示原始图像（图(a)）、使用 5×5 卷积核进行均值滤波处理结果图像（图(b)）、使用 30×30 卷积核进行均值滤波处理结果图像（图(c)），如图 7-14 所示。

从图中可以看出，使用 5×5 的卷积核进行滤波处理时，图像的失真不明显；而使用 30×30的卷积核进行滤波处理时，图像的失真情况较明显。

卷积核越大，参与到均值运算中的像素就会越多，即当前点计算的是更多点的像素值的平均值。因此，卷积核越大，去噪效果越好，当然花费的计算时间也会越长，同时让图像失真越严重。在实际处理中，要在失真和去噪效果之间取得平衡，选取合适大小的卷积核。



(a) (b) (c)

图 7-5 平滑处理像素值

1.8

高斯滤波

在进行均值滤波和方框滤波时，其邻域内每个像素的权重是相等的。在高斯滤波中，会将中心点的权重值加大，远离中心点的权重值减小，在此基础上计算邻域内各个像素值不同权重的和。

7.3.2函数语法

在 OpenCV 中，实现高斯滤波的函数是 cv2.GaussianBlur()，该函数的语法格式是：

```
dst = cv2.GaussianBlur( src, ksize, sigmaX, sigmaY, borderType )
```

式中：

- dst 是返回值，表示进行高斯滤波后得到的处理结果。
- src 是需要处理的图像，即原始图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 CV_8U、CV_16U、CV_16S、CV_32F 或者 CV_64F 中的一种。
- ksize 是滤波核的大小。滤波核大小是指在滤波处理过程中其邻域图像的高度和宽度。需要注意，滤波核的值必须是奇数。
- sigmaX 是卷积核在水平方向上（X 轴方向）的标准差，其控制的是权重比例。例如，图 7-25 中是不同的 sigmaX 决定的卷积核，它们在水平方向上的标准差不同。

7.3.3程序示例

【例 7.6】对噪声图像进行高斯滤波，显示滤波的结果。

根据题目要求，采用函数 cv2.GaussianBlur()实现高斯滤波，编写程序代码为：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.GaussianBlur(o,(5,5),0,0)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

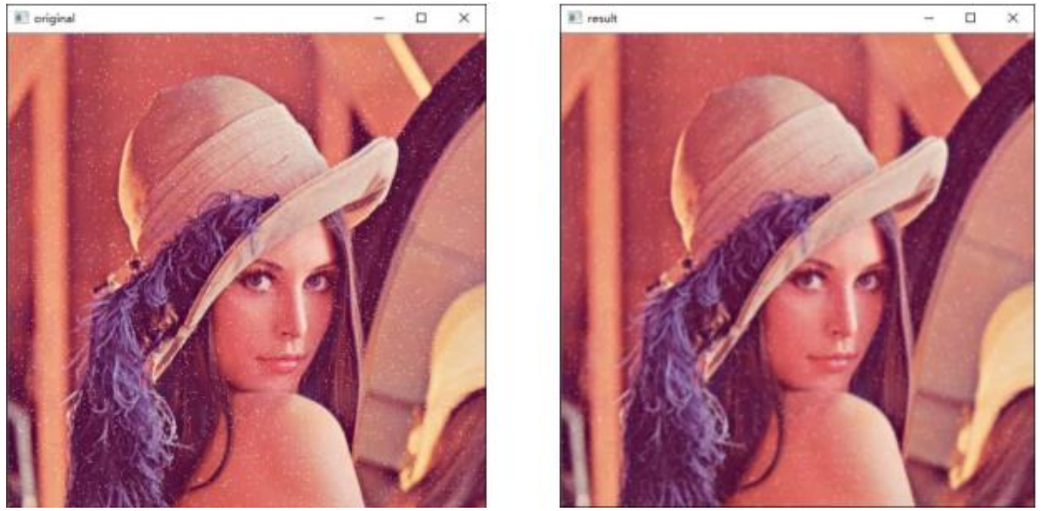


图 7-26 高斯滤波示例

运行上述程序后，结果如图 7-26 所示，其中左图是原始图像，右图是高斯滤波后的处理结果图像。

1.9

中值滤波

中值滤波与前面介绍的滤波方式不同，不再采用加权求均值的方式计算滤波结果。它用邻域内所有像素值的中间值来替代当前像素点的像素值。

7.4.2函数语法

在 OpenCV 中，实现中值滤波的函数是 cv2.medianBlur()，其语法格式如下：

```
dst = cv2.medianBlur( src, ksize)
```

式中：

- dst 是返回值，表示进行中值滤波后得到的处理结果。
- src 是需要处理的图像，即源图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 CV_8U、CV_16U、CV_16S、CV_32F 或者 CV_64F 中的一种。
- ksize 是滤波核的大小。滤波核大小是指在滤波处理过程中其邻域图像的高度和宽度。需要注意，核大小必须是比 1 大的奇数，比如 3、5、7 等。

德以明理 学以精工

7.4.3程序示例

【例 7.7】针对噪声图像，对其进行中值滤波，显示滤波的结果。

根据题目要求，采用函数 cv2.medianBlur()实现中值滤波，编写程序代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.medianBlur(o,3)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

中值滤波

运行上述程序后，结果如图 7-29 所示，其中左图是原始图像，右图是中值滤波后的处理结果图像。



图 7-29 中值滤波示例

从图中可以看到，由于没有进行均值处理，中值滤波不存在均值滤波等滤波方式带来的细节模糊问题。在中值滤波处理中，噪声成分很难被选上，所以可以在几乎不影响原有图像的情况下去除全部噪声。但是由于需要进行排序等操作，中值滤波需要的运算量较大。

1.10

腐蚀

形态学，即数学形态学（Mathematical Morphology），是图像处理过程中一个非常重要的研究方向。形态学主要从图像内提取分量信息，该分量信息通常对于表达和描绘图像的形状具有重要意义，通常是图像理解时所使用的最本质的形状特征。例如，在识别手写数字时，能够通过形态学运算得到其骨架信息，在具体识别时，仅针对其骨架进行运算即可。形态学处理在视觉检测、文字识别、医学图像处理、图像压缩编码等领域都有非常重要的应用。

形态学操作主要包含：腐蚀、膨胀、开运算、闭运算、形态学梯度（Morphological Gradient）运算、顶帽运算（礼帽运算）、黑帽运算等操作。**腐蚀操作**和**膨胀操作**是形态学运算的基础，将腐蚀和膨胀操作进行结合，就可以实现开运算、闭运算、形态学梯度运算、顶帽运算、黑帽运算、击中击不中等不同形式的运算。

腐蚀

腐蚀是最基本的形态学操作之一，它能够将图像的边界点消除，使图像沿着边界向内收缩，也可以将小于指定结构体元素的部分去除。

腐蚀用来“收缩”或者“细化”二值图像中的前景，借此实现去除噪声、元素分割等功能。例如，在图 8-1 中，左图是原始图像，右图是对其腐蚀的处理结果。

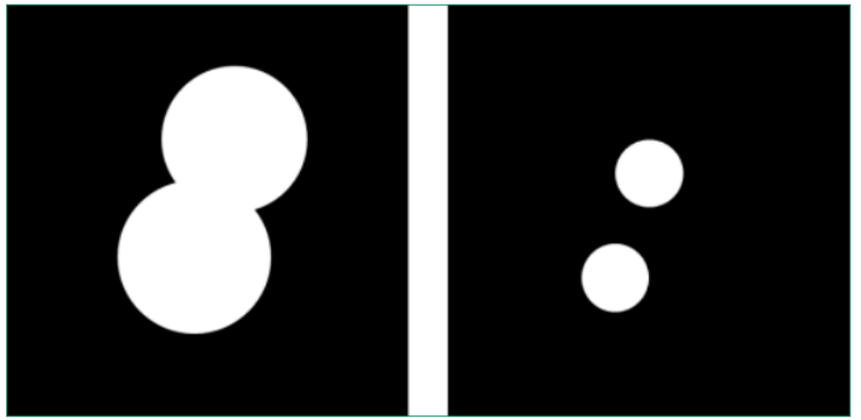


图 8-1 腐蚀示例

在腐蚀过程中，通常使用一个结构元来逐个像素地扫描要被腐蚀的图像，并根据结构元和被腐蚀图像的关系来确定腐蚀结果。

腐蚀

在 OpenCV 中，使用函数 cv2.erode()实现腐蚀操作，其语法格式为：

```
dst = cv2.erode( src, kernel[, anchor[, iterations[, borderType[,borderValue]]]] )
```

式中：

- dst 是腐蚀后所输出的目标图像，该图像和原始图像具有同样的类型和大小。
- src 是需要进行腐蚀的原始图像，图像的通道数可以是任意的。但是要求图像的深度必须是 CV_8U、CV_16U、CV_16S、CV_32F、CV_64F 中的一种。
- kernel 代表腐蚀操作时所采用的结构类型。它可以自定义生成，也可以通过函数 cv2.getStructuringElement()生成。
- anchor 代表 element 结构中锚点的位置。该值默认为(-1,-1)，在核的中心位置。
- iterations 是腐蚀操作迭代的次数，该值默认为 1，即只进行一次腐蚀操作。
- borderType 代表边界样式，一般采用其默认值 BORDER_CONSTANT。该项的具体值如表 8-1 所示。

腐蚀

【例 8.2 】使用函数 cv2.erode()完成图像腐蚀。

根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("erode.bmp",cv2.IMREAD_UNCHANGED)
kernel = np.ones((5,5),np.uint8)
erosion = cv2.erode(o,kernel)
cv2.imshow("original",o)
cv2.imshow("erosion",erosion)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，使用语句 kernel=np.ones((5,5),np.uint8)生成如图 8-5 所示的 5×5 的核，来对原始图像进行腐蚀。

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

图 8-5 自定义核

运行程序，结果如图 8-6 所示。其中，左图是原始图像，右图是腐蚀处理结果。从图中可以看到，腐蚀操作将原始图像内的毛刺腐蚀掉了。

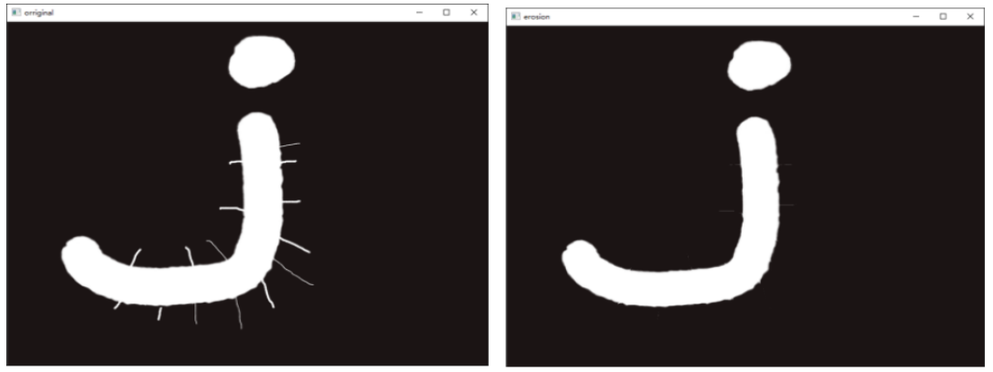


图 8-6



膨胀

膨胀

在 OpenCV 内，采用函数 cv2.dilate()实现对图像的膨胀操作，该函数的语法结构为：

```
dst = cv2.dilate( src, kernel[, anchor[, iterations[, borderType[,borderValue]]]])
```

式中：

- dst 代表膨胀后所输出的目标图像，该图像和原始图像具有同样的类型和大小。
- src 代表需要进行膨胀操作的原始图像。图像的通道数可以是任意的，但是要求图像的深度必须是 CV_8U、CV_16U、CV_16S、CV_32F、CV_64F 中的一种。
- element 代表膨胀操作所采用的结构类型。它可以自定义生成，也可以通过函数 cv2.getStructuringElement()生成。
- 参数 kernel、anchor、iterations、borderType、borderValue 与函数 cv2.erode()内相应参数的含义一致。

通用形态学函数

腐蚀操作和膨胀操作是形态学运算的基础，将腐蚀和膨胀操作进行组合，就可以实现开运算、闭运算（关运算）、形态学梯度（Morphological Gradient）运算、礼帽运算（顶帽运算）、黑帽运算、击中击不中等多种不同形式的运算。

OpenCV 提供了函数 cv2.morphologyEx()来实现上述形态学运算，其语法结构如下：

```
dst = cv2.morphologyEx( src, op, kernel[, anchor[, iterations[, borderType[,borderValue]]]]] )
```

式中：

- dst 代表经过形态学处理后所输出的目标图像，该图像和原始图像具有同样的类型和大小。
- src 代表需要进行形态学操作的原始图像。图像的通道数可以是任意的，但是要求图像的深度必须是 CV_8U、CV_16U、CV_16S、CV_32F、CV_64F 中的一种。
- op 代表操作类型，如表 8-2 所示。各种形态学运算的操作规则均是将腐蚀和膨胀操作进行组合而得到的。

先腐蚀再膨胀：开运算

开运算进行的操作是先将图像腐蚀，再对腐蚀的结果进行膨胀。开运算可以用于去噪、计数等。例如，在图 8-14 中，通过先腐蚀后膨胀的开运算操作实现了去噪，其中：

- 左图是原始图像。
- 中间的图是对原始图像进行腐蚀的结果。
- 右图是对腐蚀后的图像进行膨胀的结果，即对原始图像进行开运算的处理结果。



图 8-14 实现去噪的开运算

先腐蚀再膨胀：开运算

通过将函数 cv2.morphologyEx()中操作类型参数 op 设置为“cv2.MORPH_OPEN”，可以实现开运算。其语法结构如下：

```
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
```

【例 8.7 】使用函数 cv2.morphologyEx()实现开运算。根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img1=cv2.imread("opening.bmp")
img2=cv2.imread("opening2.bmp")
k=np.ones((10,10),np.uint8)
r1=cv2.morphologyEx(img1,cv2.MORPH_OPEN,k)
r2=cv2.morphologyEx(img2,cv2.MORPH_OPEN,k)
cv2.imshow("img1",img1)
cv2.imshow("result1",r1)
cv2.imshow("img2",img2)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()
```

先膨胀后腐蚀：闭运算

德以明理 学以精工

闭运算是先膨胀、后腐蚀的运算，它有助于关闭前景物体内部的小孔，或去除物体上的小黑点，还可以将不同的前景图像进行连接。

例如，在图 8-17 中，通过先膨胀后腐蚀的闭运算去除了原始图像内部的小孔（内部闭合的闭运算），其中：

- 左图是原始图像。
- 中间的图是对原始图像进行膨胀的结果。
- 右图是对膨胀后的图像进行腐蚀的结果，即对原始图像进行闭运算的结果。

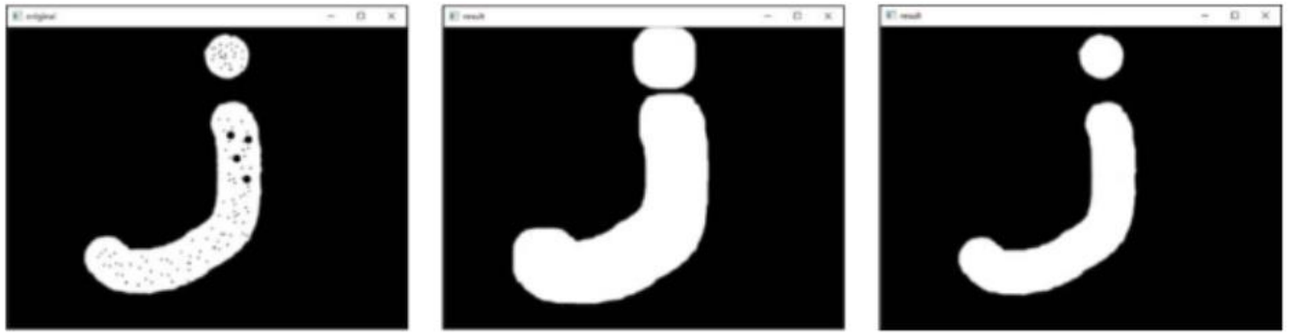


图 8-17 实现去除内部小孔的闭运算

先膨胀后腐蚀：闭运算

通过将函数 cv2.morphologyEx()中操作类型参数 op 设置为“cv2.MORPH_CLOSE”，可以实现闭运算。其语法结构如下：

```
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
```

【例 8.8 】使用函数 cv2.morphologyEx()实现闭运算。根据题目要求，编写程序如下：

```
import cv2
import numpy as np
img1=cv2.imread("closing.bmp")
img2=cv2.imread("closing2.bmp")
k=np.ones((10,10),np.uint8)
r1=cv2.morphologyEx(img1,cv2.MORPH_CLOSE,k,iterations=3)
r2=cv2.morphologyEx(img2,cv2.MORPH_CLOSE,k,iterations=3)
cv2.imshow("img1",img1)
cv2.imshow("result1",r1)
cv2.imshow("img2",img2)
cv2.imshow("result2",r2)
cv2.waitKey()
cv2.destroyAllWindows()
```

形态学梯度运算

形态学梯度运算是用图像的膨胀图像减腐蚀图像的操作，该操作可以获取原始图像中前景图像的边缘。例如，图 8-20 演示了形态学梯度运算。

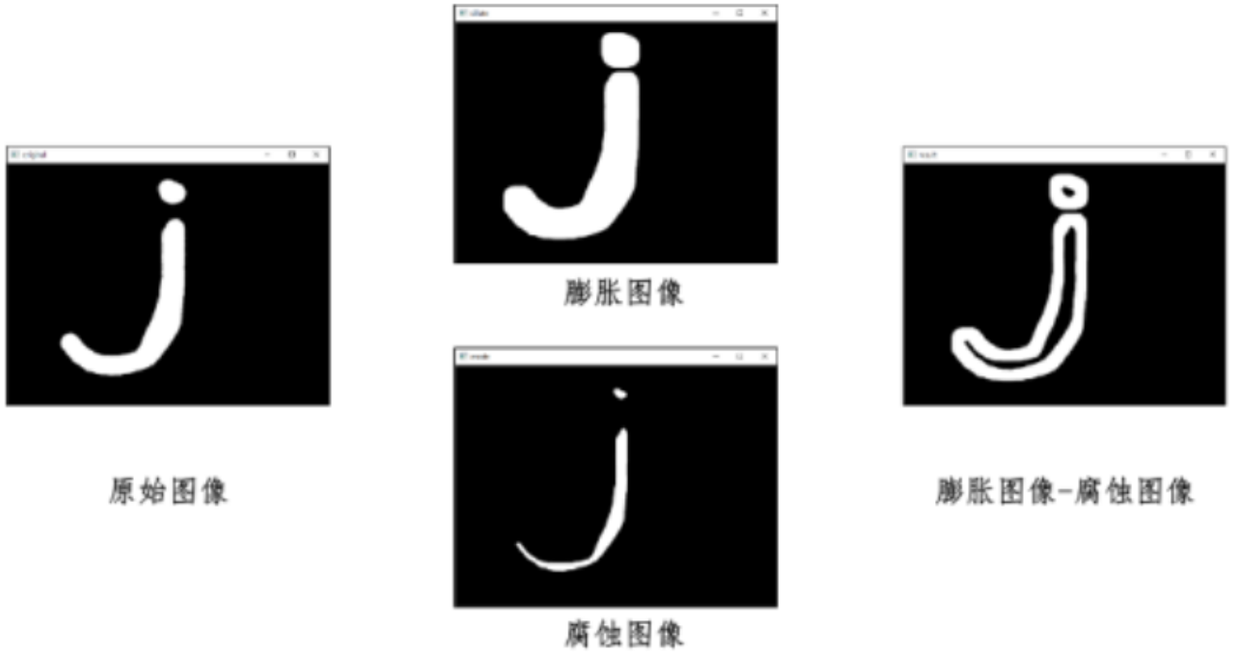


图 8-20 实现形态学梯度运算

形态学梯度运算

从图 8-20 中可以看到，形态学梯度运算使用膨胀图像（扩张亮度）减腐蚀图像（收缩亮度），得到原始图像中前景对象的边缘。

通过将函数 cv2.morphologyEx()的操作类型参数 op 设置“cv2.MORPH_GRADIENT”，可以实现形态学梯度运算。其语法结构如下：

```
result = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
```

【例 8.9 】使用函数 cv2.morphologyEx()实现形态学梯度运算。根据题目要求，编写程序如下：

```
import cv2
import numpy as np
o=cv2.imread("gradient.bmp",cv2.IMREAD_UNCHANGED)
k=np.ones((5,5),np.uint8)
r=cv2.morphologyEx(o,cv2.MORPH_GRADIENT,k)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```